

# BPMN 2.0 & EPC

Prozesse modellieren, Verantwortung sichtbar machen –  
von der Notation zur Steuerungsentscheidung.

*Lehrskript – Tag 1 von 3*

Lernpfad:  
Wissen → Anwendung → Management

Dozent:  
Can Siebert

Bearbeitungsdauer:  
1 Kurstag

Format:  
Theorie · Fallstudie · Coaching

---

## Lernziele dieses Tages

---

Am Ende dieses Tages haben Sie ein gemeinsames Vokabular für Prozessmodellierung, können BPMN 2.0 und EPC einordnen, ein erstes belastbares Modell erstellen und beginnen, Modellierungsentscheidungen aus der Management-Perspektive zu treffen. Die Lernziele sind in drei Stufen gegliedert – Wissen, Anwendung und Management – und bauen aufeinander auf.

### L1 – Wissen (Reproduktion und Einordnung)

- Begriffe Prozess, Modell, Notation, Modellzweck verstehen und voneinander abgrenzen.
- BPMN-2.0-Basiselemente kennen: Start-/End-Ereignisse, Aktivitäten/Tasks, Gateways (insb. XOR), Sequenz- und Nachrichtenfluss, Pools und Lanes.
- EPC-Basiselemente kennen: Ereignis, Funktion, Konnektoren (AND, OR, XOR), Organisationseinheit, Informationsobjekt.
- Grundsätze ordnungsmäßiger Modellierung (GoM) als Qualitätsrahmen einordnen.

### L2 – Anwendung (Transfer auf konkrete Situationen)

- Aus einer Prozessbeschreibung ein BPMN- oder EPC-Modell ableiten.
- Benennungsregeln (Verb + Objekt für Funktionen, Zustand für Ereignisse) konsistent anwenden.
- Modellqualität anhand von Kriterien bewerten und gezielt verbessern.
- Eine Notation begründet auswählen: *BPMN oder EPC für diesen Zweck?*

### L3 – Management (Entscheidung und Verantwortung)

- Modellzweck und Granularität als Managemententscheidung treffen.
- Zielkonflikte (Dokumentation vs. Automatisierung, Management vs. IT) bewusst entscheiden.
- Verantwortung über Lanes und Rollen sichtbar machen.
- Annahmen unter Unsicherheit transparent dokumentieren statt zu raten.

#### Roter Faden

Ein Prozessmodell ist kein Selbstzweck. Es ist ein *Steuerungsinstrument* – für Dokumentation, Analyse, Automatisierung oder Compliance. Wer den Zweck nicht kennt, modelliert entweder zu fein oder zu grob – und in beiden Fällen am Bedarf vorbei.

## 1. Einstieg: Warum Prozesse modellieren

Jede Organisation *hat* Prozesse – ob sie sie aufgeschrieben hat oder nicht. Bestellungen werden bearbeitet, Anträge geprüft, Geräte produziert, Tickets beantwortet. Die Frage ist nicht, *ob* es Prozesse gibt, sondern: *Sind sie verstanden, vergleichbar, steuerbar?* (vgl. Dumas et al., 2018, Kap. 1)

In der Praxis stoßen wir typischerweise auf drei Symptome, die anzeigen, dass die Prozesslandschaft modellierungsbedürftig ist:

1. **Niemand weiß, wer entscheidet.** Verantwortung liegt im Bauch einiger erfahrener Mitarbeiter – und verschwindet, sobald jemand das Unternehmen verlässt.
2. **Jeder Bereich modelliert anders – oder gar nicht.** Vergleichbarkeit fehlt, Audits werden zur Such-Aktion, IT-Schnittstellen erfindet jedes Projekt neu.
3. **Das Modell ist da – aber unbenutzbar.** Es ist zu fein, zu vage, falsch beschriftet oder seit drei Reorganisationen nicht angefasst worden.

### 1.1 Was ein Prozess ist – und was nicht

Ein **Prozess** ist eine logische Abfolge von Aktivitäten, die einen *Input* in einen *Output* transformiert und dabei einen Wert für den Kunden – intern oder extern – erzeugt. Drei Bestandteile gehören dazu:

- **Input** – Trigger oder Ausgangszustand (eine Bestellung trifft ein, eine Anforderung wird formuliert).
- **Transformation** – die eigentliche Folge von Tätigkeiten, oft mit Entscheidungen und parallelen Pfaden.
- **Output** – Ergebniszustand (eine versendete Lieferung, ein freigegebenes Produkt, ein abgelehnter Antrag).

Ein **Modell** ist nicht der Prozess. Es ist eine *Abbildung* der Realität, die bewusst Dinge weglässt, um andere Dinge sichtbar zu machen. Ein gutes Modell verschweigt Details, ohne die Logik zu verzerren.

### 1.2 Was Modellierung leistet – und was nicht

Modellierung ist nicht magisch. Sie ändert nicht die Realität, sie macht sie nur *besprechbar*. Was sie kann:

- Gemeinsames Verständnis schaffen (Fach + IT + Management sprechen über dasselbe Diagramm).
- Schwachstellen sichtbar machen (Doppelarbeit, fehlende Entscheidungen, Schleifen ohne Abbruchkriterium).
- Verantwortung dokumentieren (wer macht was, wer entscheidet).
- Eine Brücke zur Automatisierung bauen (BPMN 2.0 als executable Notation).

Was sie *nicht* kann: schlechte Prozesse durch Visualisierung gut machen. Ein modelliertes Chaos bleibt Chaos – nur dass es jetzt auf Papier steht und alle es lesen können.

#### Eselsbrücke: I-T-O

**I-T-O** – die drei Bestandteile jedes Prozesses, von links nach rechts:

**Input** – was kommt rein? (Bestellung, Antrag, Anforderung)

**Transformation** – was passiert? (prüfen, entscheiden, fertigen)

**Output** – was kommt raus? (Lieferung, Bescheid, Produkt)

*Wenn Input, Transformation oder Output unklar sind, kann man kein Modell bauen, nur eine Hoffnung skizzieren.*

### 1.3 Vier Modellierungszwecke

Bevor wir das erste Symbol zeichnen, muss *eine* Frage beantwortet sein: *Wofür modellieren wir?* In der Praxis gibt es vier Hauptzwecke, und sie verlangen unterschiedliche Tiefen, Notationen und Granularitäten:

- **Dokumentation.** Das Modell hält fest, wie ein Prozess heute läuft. Adressat: neue Mitarbeiter, Audit, Wissenssicherung. Tiefe: mittel; Klarheit wichtiger als Vollständigkeit.
- **Analyse.** Das Modell soll Schwachstellen finden – Engpässe, Schleifen, doppelte Arbeit. Adressat: Prozess-Owner, Change-Team. Tiefe: hoch, mit Mess- und Entscheidungspunkten.
- **Automatisierung.** Das Modell soll als Vorlage für eine ausführbare Lösung dienen (Workflow-Engine, Process-Mining-Tool). Adressat: IT. Tiefe: sehr hoch, jede Bedingung exakt.
- **Compliance.** Das Modell weist nach, dass regulatorische Anforderungen eingehalten werden. Adressat: Auditor, Aufsicht. Tiefe: variabel; Nachvollziehbarkeit und Versionierung sind entscheidend.

## 2. BPMN 2.0 – Basiselemente

**BPMN** steht für *Business Process Model and Notation*. Die Version 2.0 ist seit 2011 ein Standard der Object Management Group (OMG), aktuell in Version 2.0.2 von 2014. Ihr Anspruch: eine Notation, die sowohl für die Fachseite verständlich als auch für die IT ausführbar ist. In der Praxis nutzen wir nur einen Bruchteil der Spezifikation – das, was im Geschäftsalltag wirklich vorkommt. (OMG, 2014; Allweyer, 2020)

### 2.1 Aktivitäten (Activities) und Tasks

Eine **Aktivität** ist eine Tätigkeit, die innerhalb des Prozesses ausgeführt wird. Sie wird als abgerundetes Rechteck dargestellt. Die einfachste Aktivität ist der **Task** – eine atomare, nicht weiter zerlegte Tätigkeit. Komplexe Aktivitäten können als **Sub-Process** verfeinert werden (ein gekapselter Teilprozess mit eigenem Start und Ende).

Benennungsregel für Tasks: *Verb + Objekt*. Also nicht “Rechnung”, sondern “Rechnung erstellen”. Nicht “Lagerprüfung”, sondern “Lagerbestand prüfen”. Wer diese Regel konsequent durchhält, eliminiert die Hälfte aller Missverständnisse in Reviews.

### 2.2 Ereignisse (Events)

Ein **Ereignis** ist etwas, das im Prozess *passiert* – typischerweise ein Zustandswechsel oder ein Trigger. BPMN unterscheidet drei Hauptkategorien, dargestellt als Kreise:

- **Start-Ereignis** (dünner Kreis) – der Prozess beginnt. Jeder vollständige Prozess hat genau ein Start-Ereignis pro Pool (im einfachen Fall).
- **Zwischen-Ereignis** (doppelter Kreis) – etwas passiert während des Prozesses, oft ein Wartezustand (“Nachricht erhalten”, “Timer abgelaufen”).

- **End-Ereignis** (dicker Kreis) – der Prozess endet. Achtung: jeder Pfad muss zu einem End-Ereignis führen. Ein Prozess ohne klares Ende ist ein Indikator für ein unfertiges Modell.

## 2.3 Gateways

Ein **Gateway** ist eine Verzweigung oder Zusammenführung im Prozessfluss, dargestellt als Raute. Die drei wichtigsten Typen:

- **XOR-Gateway** (Raute mit “X”) – exklusive Entscheidung: genau ein Pfad wird gewählt. Beispiel: “Bestellung vollständig?” → ja oder nein.
- **AND-Gateway** (Raute mit “+”) – parallele Ausführung: alle Pfade laufen *gleichzeitig*. Wird oft zur Synchronisation am Ende wieder zusammengeführt.
- **OR-Gateway** (Raute mit “O”) – inklusiv: ein oder mehrere Pfade. Mächtig, aber kann das Modell schnell unleserlich machen – sparsam einsetzen.

### Gateway-Disziplin

Ein Gateway ist nicht dazu da, alle möglichen Pfade “aufzufächern”. Es markiert eine *echte Entscheidung*, die im Prozess so gefällt wird. Wenn vor dem Gateway nicht klar ist, wer auf welcher Basis entscheidet, ist das Gateway eine Lüge – es täuscht Kontrolle vor, wo Improvisation herrscht.

## 2.4 Sequenz- und Nachrichtenflüsse

Aktivitäten, Ereignisse und Gateways werden über **Sequenzflüsse** verbunden (durchgezogener Pfeil). Sie zeigen die zeitliche Reihenfolge *innerhalb* eines Pools. Zwischen verschiedenen Pools (also zwischen unterschiedlichen Beteiligten) wird der **Nachrichtenfluss** verwendet (gestrichelter Pfeil mit offener Spitze). Er signalisiert: hier wird kommuniziert, hier verlässt etwas die Organisationsgrenze.

## 2.5 Pools und Lanes

Ein **Pool** steht für einen Beteiligten – meist eine ganze Organisation oder einen organisatorisch eigenständigen Bereich. Innerhalb eines Pools gliedern **Lanes** weiter nach Rollen, Abteilungen oder Systemen. Beispiel: Pool “NovaParts GmbH”, Lanes “Vertrieb”, “Produktion”, “Versand”, “Buchhaltung”.

Lanes sind das wichtigste *Verantwortungs-Werkzeug* der BPMN. Eine Aktivität in einer Lane heißt: *Diese Rolle führt diese Tätigkeit durch*. Damit ist Verantwortung nicht mehr Bauchwissen, sondern Diagramm-Eigenschaft.

## 2.6 Artifacts

**Artifacts** sind ergänzende Elemente, die den Prozess kontextualisieren, ohne ihn zu verändern: *Data Objects* (z. B. “Bestellung als PDF”), *Annotations* (Kommentare am Diagramm), *Groups* (visuelle Gruppierung). Sie helfen der Lesbarkeit, sind aber kein Pflichtprogramm.

### Eselsbrücke: A-E-G-F-P

Die fünf BPMN-Hauptelemente, die in 90 % der Modelle vorkommen:

Aktivitäten – was wird getan?

Ereignisse – was passiert? (Start, Zwischen, Ende)

Gateways – wo wird entschieden?

Flüsse – wie hängt es zusammen? (Sequenz, Nachricht)

Pools/Lanes – wer ist verantwortlich?

Mit “A-E-G-F-P” kann man fast jedes Prozessdiagramm bauen – alles andere ist Feinschliff.

### 3. EPC – Ereignisgesteuerte Prozesskette

**EPC** steht für *Ereignisgesteuerte Prozesskette*. Sie wurde Anfang der 1990er Jahre am Institut für Wirtschaftsinformatik in Saarbrücken (Scheer) entwickelt und ist in deutschsprachigen Organisationen weit verbreitet – besonders in SAP-nahen Umgebungen und in Verwaltungen. Wer im DACH-Raum mit gewachsenen Prozesslandschaften zu tun hat, trifft EPC garantiert. (Scheer, 2002)

#### 3.1 Die drei Pflichtelemente

EPC ist im Kern denkbar einfach. Drei Symbole tragen die Notation:

- **Ereignis** (Sechseck) – ein Zustand. Beispiel: “Bestellung eingegangen”, “Lager geprüft”, “Rechnung erstellt”. Benennung passiv: Zustand, der eingetreten ist.
- **Funktion** (abgerundetes Rechteck) – eine Tätigkeit. Beispiel: “Lagerbestand prüfen”, “Rechnung erstellen”. Benennung aktiv: Verb + Objekt.
- **Konnektor** (Kreis mit Symbol) – logische Verknüpfung: AND ( $\wedge$ ), OR ( $\vee$ ), XOR.

Die zentrale Spielregel der EPC: *Ereignis und Funktion wechseln sich strikt ab*. Auf ein Ereignis folgt eine Funktion, auf eine Funktion ein Ereignis. Diese Disziplin macht EPC besonders gut lesbar – und besonders streng.

#### 3.2 Ergänzende Elemente

In erweiterten EPC (eEPC) kommen zwei weitere Elemente häufig vor:

- **Organisationseinheit** (Ellipse) – wer führt die Funktion aus? (Abteilung, Rolle, Stelle).
- **Informationsobjekt** (Rechteck) – welche Information wird benötigt oder erzeugt? (Bestellung, Kundenstammsatz, Lagerbestand).

#### 3.3 BPMN vs. EPC – wann was

Beide Notationen können denselben Prozess abbilden – aber mit unterschiedlichen Schwerpunkten:

|                                 | BPMN 2.0                                       | EPC                                                    |
|---------------------------------|------------------------------------------------|--------------------------------------------------------|
| <b>Schwerpunkt</b>              | Ablauflogik, Rollen, Automatisierung           | Ereignisgetriebene Wertschöpfungskette, Business-Sicht |
| <b>Verbreitung</b>              | internationaler Standard (OMG)                 | DACH, oft SAP-/ARIS-Umfeld                             |
| <b>Nähe zur IT</b>              | hoch – executable BPMN möglich                 | gering – Beschreibungsmodell                           |
| <b>Lesbarkeit für Fachseite</b> | gut bei einfachen Modellen                     | sehr gut, klare Sprache                                |
| <b>Rollendarstellung</b>        | über Pools/Lanes nativ                         | über Organisationseinheiten (optional)                 |
| <b>Skalierbarkeit</b>           | komplexe Modelle möglich, aber unübersichtlich | gut für Übersicht, weniger für Tiefe                   |

**Faustregel.** Geht es Richtung Automatisierung oder Schnittstelle zur IT? → BPMN. Geht es um eine Management- oder Fachübersicht, in der Ereignisse den Takt geben? → EPC. In hybriden Landschaften: EPC für die Übersicht, BPMN für die Detailebene, die nahe an der IT liegt.

#### Notation als Werkzeug, nicht als Religion

Die Frage “BPMN oder EPC” ist meist falsch gestellt. Die richtige Frage lautet: *Für welchen Zweck, für welche Zielgruppe, mit welcher Tiefe modellieren wir?* Wer den Zweck kennt, wählt die Notation als Werkzeug – und nicht aus Tool-Gewohnheit.

## 4. Modell-Qualität: die sieben Grundsätze

Ein Modell ist nicht “richtig” oder “falsch” – es ist mehr oder weniger *brauchbar* für seinen Zweck. Die **Grundsätze ordnungsmäßiger Modellierung** (GoM) – ursprünglich von Becker, Rosemann und Schütte (1995) formuliert – geben sieben Kriterien an die Hand, die in jeder Modellierungs-Disziplin gelten. (Becker et al., 1995) Eine empirisch fundierte, kompaktere Schwester-Heuristik sind die “Seven Process Modeling Guidelines” (7PMG) von Mendling, Reijers und van der Aalst – sie übersetzen die GoM in konkrete Daumenregeln für die tägliche Modellierungspraxis. (Mendling et al., 2010)

### 4.1 Die sieben Grundsätze im Detail

1. **Grundsatz der Richtigkeit.** Das Modell bildet die Realität (oder das gewünschte Soll) *korrekt* ab. Es enthält keine Widersprüche, die Logik trägt. Test: lässt sich der Prozess so wie modelliert ausführen?
2. **Grundsatz der Relevanz.** Das Modell enthält nur das, was für seinen Zweck relevant ist. Alles andere lenkt ab. Test: kann ich ein Element streichen, ohne dass die Zweckaussage leidet?
3. **Grundsatz der Wirtschaftlichkeit.** Aufwand für Modellierung und Pflege steht in einem vernünftigen Verhältnis zum Nutzen. Test: lohnt sich das Detail noch, oder sind wir im Modellierungs-Selbstzweck?
4. **Grundsatz der Klarheit.** Das Modell ist für die Zielgruppe *lesbar* – nicht für den Modellierer selbst. Test: versteht eine Person aus der Zielgruppe es ohne mündliche Erklärung?
5. **Grundsatz der Vergleichbarkeit.** Das Modell folgt denselben Konventionen wie andere Modelle derselben Organisation: Benennung, Granularität, Symbolik. Test: kann ich es

neben ein Nachbarmodell legen und beide vergleichen?

6. **Grundsatz des systematischen Aufbaus.** Das Modell ist konsistent strukturiert – Hierarchie, Schichten, Schnittstellen sind nachvollziehbar. Test: gibt es ein erkennbares Muster, oder ist es Wildwuchs?
7. **Grundsatz der Zweckorientierung.** Das Modell dient einem klar benannten Zweck. Alle anderen Grundsätze sind ihm untergeordnet. Test: kann ich in einem Satz sagen, wofür dieses Modell gebraucht wird?

## 4.2 Die sieben Grundsätze als Checkliste

|                              |                                                           |
|------------------------------|-----------------------------------------------------------|
| <b>Richtigkeit</b>           | Logik schlüssig, keine Widersprüche, ausführbare Pfade.   |
| <b>Relevanz</b>              | Nur das Notwendige – kein Selbstzweck-Detail.             |
| <b>Wirtschaftlichkeit</b>    | Aufwand < Nutzen, auch in der Pflege.                     |
| <b>Klarheit</b>              | Lesbarkeit für die Zielgruppe, nicht für den Modellierer. |
| <b>Vergleichbarkeit</b>      | Konventionen einhalten, neben Nachbarmodellen lesbar.     |
| <b>Systematischer Aufbau</b> | Konsistente Hierarchie und Schnittstellen.                |
| <b>Zweckorientierung</b>     | Ein klar benannter Zweck – alles andere folgt.            |

### Eselsbrücke: R-R-W-K-V-S-Z

“Richtig, relevant, wirtschaftlich, klar, vergleichbar, systematisch, zweckorientiert.”

Sieben Buchstaben, sieben Tests vor jeder Modell-Freigabe. Wer fünf von sieben erfüllt, hat ein gutes Modell. Wer Zweckorientierung verfehlt, hat — egal wie schön gezeichnet — kein brauchbares.

## 5. Zweckorientierung: was VOR dem Modellieren zu klären ist

Der wichtigste der sieben Grundsätze – Zweckorientierung – bekommt einen eigenen Abschnitt, weil er in der Praxis am häufigsten übersprungen wird. Modellierer sind oft so erpicht, ein Tool zu öffnen und mit Symbolen anzufangen, dass sie die drei wichtigen Fragen nicht stellen.

### 5.1 Drei Fragen vor jedem Modell

1. **Wofür?** (Modellzweck) – Dokumentation, Analyse, Automatisierung, Compliance? Häufig ist es eine *Hauptzweck-plus-Nebennutzen*-Konstellation. Sie muss benannt sein.
2. **Für wen?** (Zielgruppe) – Fachseite, Management, IT, Auditor, neue Mitarbeiter? Die Zielgruppe bestimmt Sprache, Tiefe und Symbolik.
3. **Wie tief?** (Granularität) – Übersicht über das gesamte Geschäftsfeld, oder Detail bis auf Klick-Ebene? Tiefe ist eine Entscheidung, kein Zufall.

### 5.2 Granularität als Managemententscheidung

Granularität ist der häufigste Stolperstein. “Zu fein” bedeutet: niemand findet die wichtige Information, Pflegeaufwand explodiert, Modelle altern schnell. “Zu grob” bedeutet: das Modell sagt nichts Verwertbares, Entscheidungen müssen woanders getroffen werden. Beide Fehler kosten – in unterschiedlicher Form.

Eine praktikable Heuristik: *Die kleinste sinnvolle Einheit ist eine Aktivität, die immer von derselben Rolle, im gleichen System, ohne Unterbrechung durchgeführt wird.* Wechselt die Rol-



le, das System oder der Zeitpunkt – ist es eine neue Aktivität.

### 5.3 Stakeholder identifizieren

Wer wird das Modell *tatsächlich nutzen*? Diese Frage trennt Modelle, die in Schubladen sterben, von solchen, die im Alltag genutzt werden:

- **Prozess-Owner** – verantwortet das End-to-End-Ergebnis und die Optimierung.
- **Beteiligte Rollen** – führen einzelne Aktivitäten aus, geben Realitäts-Feedback.
- **IT/Entwicklung** – bei Automatisierungs- oder Schnittstellen-Bezug.
- **Management** – nutzt Modelle für Priorisierung, Investitionsentscheidungen, Reporting.
- **Compliance/Audit** – benötigt Nachvollziehbarkeit und Versionierung.

#### Modellzweck-Statement – ein Satz, der trägt

Vor jedem Modell sollte *ein* Satz stehen, der drei Lücken füllt:

“Dieses Modell beschreibt den Prozess \_\_\_\_\_, für die Zielgruppe \_\_\_\_\_, mit dem Zweck \_\_\_\_\_.”

Wer diesen Satz nicht aufschreiben kann, sollte nicht modellieren – sondern fragen.

## 6. Pools, Lanes und Verantwortlichkeiten

Ein BPMN-Diagramm ohne Pools und Lanes ist eine Folge bunter Symbole – aber kein Verantwortungsmodell. Erst die Aufteilung in Pools (Beteiligte) und Lanes (Rollen) macht ein Diagramm steuerungstauglich. (Freund & Rücker, 2019, Kap. 3)

### 6.1 Pools: organisatorische Grenzen

Ein **Pool** markiert eine organisatorisch eigenständige Einheit – die eigene Firma, einen externen Dienstleister, einen Kunden. Was über die Pool-Grenze geht, ist eine *Schnittstelle*: ein Vertrag, eine Datenübertragung, ein Telefonat. Diese Schnittstellen sind oft die teuersten Stellen im Prozess – weil hier Wartezeit, Missverständnisse und Verantwortungslücken entstehen.

In BPMN werden Pools nebeneinander dargestellt, durch **Nachrichtenflüsse** (gestrichelt) verbunden. Innerhalb eines Pools fließt die Logik über **Sequenzflüsse** (durchgezogen).

### 6.2 Lanes: Rollen statt Personen

Innerhalb eines Pools strukturieren **Lanes** die Verantwortung. Wichtig: Lanes stehen für *Rollen*, nicht für konkrete Personen. “Vertrieb” ist eine Lane, nicht “Frau Meier”. Wenn Frau Meier morgen geht, bleibt die Lane bestehen, nur der Bewohner wechselt.

Typische Lanes in einem Vertriebs-/Produktionsprozess: *Vertrieb, Einkauf, Produktion, Qualität, Versand, Buchhaltung*. Typische Lanes in einem Innovationsprozess: *Produktmanagement, F&E, Qualität, Operations*, optional *Legal* und *Einkauf*.

### 6.3 Verantwortung sichtbar machen – RACI an der Lane

Lanes lassen sich gut mit einer **RACI**-Matrix kombinieren: für jede Aktivität wird festgelegt, wer *Responsible* (führt aus), *Accountable* (verantwortet), *Consulted* (wird gefragt) und *Informed* (wird informiert) ist. Die Lane zeigt typischerweise die *R*; die anderen Rollen werden in einer ergänzenden Tabelle gepflegt. (PMI, 2021)

### Verantwortungs-Test

Wenn man ein BPMN-Modell drucken kann, ohne die Lanes – und niemand bemerkt einen Verlust – ist das Modell als Verantwortungsdokument nutzlos. Lanes sind die Stelle, an der aus einem *Ablaufbild* ein *Steuerungsmodell* wird.

## 6.4 Häufige Fehler bei Pools und Lanes

- **Zu viele Lanes.** Wenn jedes Modell zehn Lanes hat, wird das Diagramm unleserlich. Zusammenfassen, wo möglich – Detail in Sub-Prozesse verlagern.
- **Externe Beteiligte als Lane.** Kunden, Lieferanten, Behörden gehören in einen *eigenen Pool*, nicht in eine Lane des eigenen Pools. Sonst wird die organisatorische Grenze unsichtbar.
- **Eine Aktivität in zwei Lanes.** Eine Aktivität gehört zu *einer* Rolle. Wenn zwei zusammen handeln, ist es entweder eine Übergabe (zwei Aktivitäten) oder eine ungenaue Beschreibung.

## 7. Häufige Fehler in Praxis-Modellen

Wer Prozessmodelle reviewt, sieht in 80 % der Fälle dieselben sechs Fehlerbilder. Wer sie kennt, kann sie vermeiden – und ein eigenes Modell selbst prüfen, bevor andere es zerlegen.

### 7.1 Sechs typische Anti-Muster

1. **Spaghetti-Modell.** 40 Pfeile kreuzen sich, das Diagramm passt nicht mehr auf eine Seite, niemand versteht es. Ursache: keine Hierarchie. Heilmittel: Sub-Prozesse, Layering, weniger Detail.
2. **Fehlende End-Ereignisse.** Mehrere Pfade verlieren sich im Nichts. Ein Modell ohne klare Enden ist unvollständig – und kein *vollständiges* Modell. Heilmittel: jedem Pfad ein End-Ereignis zuordnen, auch dem “abgelehnt”-Pfad.
3. **Falsche Gateway-Logik.** Auf ein XOR-Gateway folgen drei Pfade, die sich überlappen – oder ein AND-Gateway, das nie wieder synchronisiert wird. Heilmittel: Gateways immer paarig denken (öffnen + schließen), Bedingungen disjunkt formulieren.
4. **Inkonsistente Benennung.** “Bestellung prüfen” neben “Prüfung Bestellung” neben “Bestellatenkontrolle”. Drei Begriffe, vermutlich dieselbe Tätigkeit. Heilmittel: Glossar, einheitliche Regel Verb + Objekt.
5. **Endlosschleifen ohne Abbruchkriterium.** “Nacharbeiten” → “erneut prüfen” → “Nacharbeiten” – und niemand weiß, nach wie vielen Iterationen es eskaliert. Heilmittel: Schleifen mit klarem Abbruchkriterium oder Eskalationspfad.
6. **Modell ohne Annahmen-Hinweis.** Im Diagramm steht eine Aktivität “automatisch übermitteln” – aber niemand weiß, ob die Schnittstelle existiert. Heilmittel: Annahmen explizit dokumentieren, z. B. als Annotation.

### 7.2 Review-Checkliste in fünf Punkten

1. Hat jeder Pfad ein klares Ende?
2. Sind alle Gateways Entscheidungen, die jemand tatsächlich trifft?
3. Folgen alle Benennungen einer Regel (Verb + Objekt)?
4. Ist Verantwortung über Lanes sichtbar?

## 5. Sind Annahmen unter Unsicherheit explizit gemacht?

### Spaghetti-Test

Wenn man den Modellausdruck einer Kollegin auf den Tisch legt und sie nach 60 Sekunden *nicht* sagen kann, worum es geht, ist das Modell nicht zu klein gedruckt – es ist nicht gut genug. Das ist kein Druck-Problem, sondern ein *Klarheits*-Problem.

## 8. Vom Diagramm zur Ausführung: BPMN als Brücke

Eine Besonderheit der BPMN 2.0 ist, dass sie nicht nur dokumentieren, sondern auch *ausführen* kann. Mit einer **Workflow-Engine** (Camunda, Flowable, jBPM und andere) lassen sich BPMN-Diagramme direkt als Software-Prozess betreiben – mit Tasks, die User zugewiesen oder von Services automatisch ausgeführt werden. Das ist der Brückenkopf zwischen Fach und IT. (Freund & Rücker, 2019; Dumas et al., 2018, Kap. 9)

### 8.1 Was “executable BPMN” bedeutet

Ein ausführbares BPMN-Modell unterscheidet sich vom rein dokumentarischen Modell in mehreren Punkten:

- **Vollständigkeit.** Jeder Pfad muss durchführbar sein – es darf keine “schwebenden” Aktivitäten geben.
- **Eindeutigkeit der Gateways.** Bedingungen müssen formal beschrieben sein (oft in **FEEL**, der Decision-Engine-Sprache, oder in einer Skriptsprache).
- **Task-Typisierung.** BPMN unterscheidet *User Task* (ein Mensch arbeitet), *Service Task* (ein System arbeitet), *Send/Receive Task* (Nachrichtenaustausch), *Script Task* (kleine Logik), *Business Rule Task* (Entscheidungslogik).
- **Datenobjekte und Variablen.** Was geht in den Prozess hinein, was kommt heraus, was wird zwischen Tasks weitergegeben?

### 8.2 Strategische Bedeutung: BPMN als Übersetzer

Der eigentliche Wert von BPMN 2.0 liegt darin, dass *ein und dieselbe Notation* von Fach und IT genutzt werden kann. Die Fachseite zeichnet einen Prozess – die IT-Seite hängt Service-Tasks an die richtigen Stellen. Wer das gut hinbekommt, vermeidet die klassische Brüche zwischen “so wollten wir’s” und “so haben wir’s gebaut”.

In der Praxis ist diese Brücke nicht voraussetzungsfrei: sie erfordert Disziplin in der Modellierung, ein gemeinsames Glossar und ein klares Bekenntnis, dass das BPMN-Modell die *single source of truth* ist – nicht das Word-Dokument daneben.

### 8.3 Tool-Landschaft – ein Überblick

Ohne Empfehlung im Detail (Tooling ist projekt- und budgetabhängig) – diese Kategorien sind 2026 relevant:

- **Reine Modellierungs-Tools** (z. B. Camunda Modeler als Desktop, draw.io, bpmn.io). Geringe Hürde, gut für Dokumentation und Analyse.
- **ARIS-/Signavio-Klasse.** Enterprise-Modellierungs-Plattformen mit Governance, Versionierung, Repository.
- **Workflow-Engines** (Camunda Platform, Flowable, jBPM). Führen BPMN aus, integrieren

mit Microservices und User-Interfaces.

- **Low-Code/No-Code-Plattformen**, die BPMN-ähnliche Notationen für schnelle Workflow-Automatisierung anbieten – mit Tendenz zur Vereinfachung der Notation.

## 8.4 Wann nicht automatisieren

Nicht jeder dokumentierte Prozess ist ein Kandidat für Automatisierung. Drei Indikatoren, die zur Zurückhaltung mahnen:

- **Hohe Varianz pro Fall.** Wenn fast jeder Vorgang anders ist, wird der Workflow zur Sammlung von Sonderlocken.
- **Kurze Lebensdauer des Prozesses.** Wenn die Organisation den Prozess in zwölf Monaten neu erfindet, lohnt sich die Investition nicht.
- **Niedrige Frequenz.** Ein Prozess, der dreimal im Jahr läuft, ist ein Kandidat für eine Checkliste, nicht für eine Engine.

## 9. Coaching: Modell vs. Realität

In den ersten Sektionen ging es um Notation und Qualität – die handwerkliche Seite. In diesem Abschnitt geht es um eine andere Frage, die in jedem Modellierungsprojekt früher oder später aufkommt: *Wie genau bilden wir die Realität ab – und wessen Realität ist das eigentlich?* Das sind die Coaching-Themen, die im Tag aus UE 4 stammen.

### 9.1 Wofür modellieren wir wirklich?

Auf der Oberfläche ist die Antwort einfach (siehe Sektion 5): Dokumentation, Analyse, Automatisierung, Compliance. In der Tiefe ist sie politischer. Modelle haben Nebenwirkungen:

- Wer im Modell als “Entscheider” eingezeichnet ist, bekommt diese Rolle de facto verliehen – ob er sie wollte oder nicht.
- Wer im Modell *nicht* vorkommt, wird in spätere Optimierungen oft auch nicht einbezogen.
- Wer das Modell pflegt, prägt, wie der Prozess wahrgenommen wird – unabhängig davon, wie er tatsächlich läuft.

Senior-Modellierung bedeutet, diese Nebenwirkungen mitzudenken. Ein Modell ist ein *Macht-instrument*, nicht nur ein Diagramm.

### 9.2 Granularität bewusst entscheiden

Wie genau wir modellieren, ist eine Entscheidung – und sie hat Kosten in beide Richtungen. Drei Leitfragen:

1. Welche Modellierungs-Entscheidung von heute hat den größten Einfluss auf spätere Kosten und Risiko?
2. Welche Information fehlt – und wie würde ich sie mit minimalem Aufwand beschaffen?
3. Wo würde ich bewusst *nicht* weiter detaillieren – und warum?

### 9.3 Umgang mit Unklarheit – Annahmen sichtbar machen

In der Praxis sind viele Informationen unvollständig: Schnittstellen unbekannt, Zuständigkeiten umstritten, Fallzahlen geschätzt. Der häufigste Fehler ist es, diese Lücken durch *Raten* zu schließen, ohne es kenntlich zu machen. Das Modell sieht dann sauber aus – ist aber innen voller Annahmen, die niemand kennt.

Die Senior-Disziplin: *Annahmen explizit machen*. Eine Annotation am Diagramm (“Annahme: Schnittstelle XY existiert, zu validieren bis KW 28”), ein Annahmen-Logbuch neben dem Modell, eine Liste offener Klärungen. Transparenz ist hier kein Schwäche-Eingeständnis – sie ist die Form, in der Verantwortung sichtbar wird.

## 9.4 Entscheidungslogik: Wo muss ein Gateway hin?

Nicht jede “Wenn-dann”-Konstruktion gehört in ein Modell als Gateway. Manche Verzweigungen sind nur *Beschreibungsdetails*, andere sind echte Steuerungspunkte. Faustregel:

- **Echtes Gateway:** an dieser Stelle entscheidet eine Person oder Regel mit nachvollziehbaren Kriterien – und die Wahl hat erkennbare Konsequenzen für den weiteren Pfad.
- **Beschreibungsdetail:** an dieser Stelle gibt es zwar Varianten, aber sie werden situativ entschieden und sind nicht entscheidungsrelevant für die Steuerung des Gesamtprozesses.

## 9.5 Senior-Shift: Verantwortungsfähigkeit statt Mod-Tooling

Der Wechsel vom Modellierer zur *Verantwortungsfähigkeit* ist der eigentliche Sprung im Tag. Drei Kennzeichen, an denen man diesen Wechsel erkennt:

- Man modelliert *vor* dem Werkzeug – also: man kennt Zweck und Zielgruppe, bevor man das Tool öffnet.
- Man ist bereit, Modelle bewusst *unvollständig* zu lassen, mit dokumentierten Lücken, statt sie scheinbar genau zu machen.
- Man führt eine Diskussion über das Modell mit dem Vorstand in 90 Sekunden – nicht über Notation, sondern über die Steuerungslogik.

### Eselsbrücke: Z-G-A-V

Vier Senior-Hebel, die in jeder Modellierungs-Diskussion zuerst geklärt werden müssen:

**Zweck** – wofür modellieren wir?

**Granularität** – wie tief?

**Annahmen** – was wissen wir nicht (und wie machen wir’s sichtbar)?

**Verantwortung** – wer entscheidet, wer liefert zu?

“Z-G-A-V” — vier Fragen, die vor jedem Diagramm-Symbol beantwortet sein sollten.

## 10. Management-Perspektive: Modellierung als Architektur

Auf Wissens-Ebene ist Modellierung eine Notation. Auf Anwendungs-Ebene ist sie ein Handwerk. Auf Management-Ebene ist sie eine *Architekturentscheidung*: Welche Prozesse modellieren wir überhaupt? Wie tief? Mit welcher Governance? Wer prüft, wer entscheidet, wer hält den Standard?

### 10.1 Zwei zentrale Zielkonflikte

1. **Schnell dokumentieren vs. später automatisieren können.** Ein hastig gezeichnetes Übersichtsmodell schafft Klarheit – ist aber selten direkt automatisierbar. Wer auf Automatisierung zielt, muss von Anfang an formal arbeiten. Das kostet Zeit, die man am Anfang nicht hat.
2. **Für Management verständlich vs. für IT ausführbar.** Management will eine A4-Übersicht. IT will jede Bedingung formal. Beide Anforderungen in *einem* Modell zu erfüllen, ist eine

Lebenslüge. Praktikabel ist eine *Schichtarchitektur*: oben EPC-/BPMN-Übersicht, darunter detaillierte BPMN-Sub-Prozesse.

## 10.2 Governance: Modellierung im Mehr-Bereichs-Kontext

Sobald mehrere Bereiche modellieren, entstehen typische Probleme:

- Jeder Bereich benennt anders – Vergleichbarkeit verschwindet.
- Niemand prüft Modelle – Qualität verfällt.
- Modelle altern – der Stand spiegelt nicht mehr die Realität.
- Tools wuchern – jeder zeichnet, wo er Lust hat.

Eine **Modellierungs-Policy** (1 Seite, nicht 20) regelt das Minimum: Wann BPMN, wann EPC? Welche Benennungsregeln? Welche Mindest-Qualität (Start/Ende, Gateways, Lanes)? Wer reviewt? Wann wird aktualisiert?

## 10.3 Rollen in einem Governance-Modell

Eine pragmatische Rollenverteilung in einer wachsenden Organisation:

- **Process Owner** – verantwortet einen End-to-End-Prozess fachlich, vertritt ihn nach außen.
- **Modeler** – modelliert konkrete Prozesse, kennt das Handwerk.
- **Reviewer** – prüft Modelle gegen die Policy und gegen GoM.
- **Approver** – gibt finale Modellversion frei (oft Process Owner oder Bereichsleiter).

Diese vier Rollen lassen sich als RACI darstellen und im Modell selbst über Lanes verankern.

## 10.4 Risiken bei Modellierungs-Initiativen

1. **Übermodellierung.** Alles wird in jedem Detail gezeichnet – Aufwand verschlingt Nutzen.
2. **Schattenprozesse.** Das Modell stimmt nicht mehr mit der Wirklichkeit überein; die Wirklichkeit läuft daneben.
3. **Akzeptanzproblem.** Die operativen Teams empfinden Modellierung als “Aufgabe von oben” – ohne erkennbaren Nutzen für sie selbst.
4. **Tool-Falle.** Diskussionen drehen sich um das Tool statt um den Prozess.
5. **Audit-Fokus.** Modellierung wird nur für Compliance gemacht – nicht für Steuerung. Das Ergebnis erfüllt den Audit, schafft aber keinen operativen Wert.

### Schluss-Merksatz für Tag 1

Prozessmodellierung ist eine **Architekturentscheidung**, kein Zeichen-Hobby. Sie wirkt jahrelang, sie verändert Sprache, Verantwortlichkeiten und Schnittstellen, und sie kann durch saubere Methoden *strukturiert*, aber nicht algorithmisch *ersetzt* werden. *Wer modelliert, entscheidet — wer entscheidet, verantwortet.*

## 11. Take-aways aus Tag 1

1. **Prozess = Input → Transformation → Output.** Wenn ein Bestandteil unklar ist, kann man kein Modell bauen, nur eine Vermutung skizzieren.
2. **Modell-Zweck zuerst, Symbol danach.** Dokumentation, Analyse, Automatisierung oder Compliance – jeder Zweck verlangt andere Tiefe und Notation.

3. **BPMN nahe IT, EPC nahe Business** – aber im Zweifel entscheidet der Zweck, nicht die Tool-Gewohnheit.
4. **Sieben GoM-Grundsätze als Checkliste:** Richtigkeit, Relevanz, Wirtschaftlichkeit, Klarheit, Vergleichbarkeit, systematischer Aufbau, Zweckorientierung.
5. **Lanes sind das Verantwortungs-Werkzeug.** Ohne Lanes ist BPMN kein Steuerungsmodell.
6. **Gateways nur, wo es echte Entscheidungen gibt.** Jede zusätzliche Raute kostet Klarheit – sie muss sich rechtfertigen.
7. **Annahmen sichtbar machen.** Transparenz unter Unsicherheit ist die senior-tauglichste Form von Verantwortung.
8. **Modellierung ist Architektur.** Im Mehr-Bereichs-Kontext entscheidet Governance darüber, ob Modelle lebendig bleiben oder im Repository sterben.

## 12. Literatur

---

Im Skript verwendete und für die Vertiefung empfohlene Quellen. Zitierstil: APA-7.

**Allweyer, T. (2020).**

*BPMN 2.0 – Business Process Model and Notation: Einführung in den Standard für die Geschäftsprozessmodellierung* (4. Aufl.). BoD.

**Becker, J., Rosemann, M., & Schütte, R. (1995).**

Grundsätze ordnungsmäßiger Modellierung. *Wirtschaftsinformatik*, 37(5), 435–445.

**Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018).**

*Fundamentals of business process management* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>

**Freund, J., & Rücker, B. (2019).**

*Real-life BPMN: Includes an introduction to DMN* (4th ed.). CreateSpace.

**Mendling, J., Reijers, H. A., & van der Aalst, W. M. P. (2010).**

Seven process modeling guidelines (7PMG). *Information and Software Technology*, 52(2), 127–136. <https://doi.org/10.1016/j.infsof.2009.08.004>

**Object Management Group. (2014).**

*Business Process Model and Notation (BPMN), Version 2.0.2* (formal/2013-12-09). <https://www.omg.org/spec/BPMN/2.0.2/>

**Project Management Institute. (2021).**

*A guide to the Project Management Body of Knowledge (PMBOK® Guide)* (7th ed.). Project Management Institute.

**Scheer, A.-W. (2002).**

*ARIS – Vom Geschäftsprozess zum Anwendungssystem* (4. Aufl.). Springer.

**van der Aalst, W. M. P. (2016).**

*Process mining: Data science in action* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-49851-4>



## 13. Glossar

---

Die wichtigsten Begriffe des Tages – kurz definiert, in alphabetischer Reihenfolge.

### **Activity / Task**

Tätigkeit im BPMN-Prozess. Atomar als Task, zerlegt als Sub-Process. Benennung: Verb + Objekt.

### **BPMN 2.0**

*Business Process Model and Notation*, Version 2.0. OMG-Standard für Prozessmodellierung. Geeignet für Fach und IT (executable BPMN).

### **EPC**

*Ereignisgesteuerte Prozesskette*. Im DACH-Raum verbreitete Notation mit strikt alternierenden Ereignissen und Funktionen.

### **Ereignis (Event)**

Zustand oder Trigger im Prozess. In BPMN als Kreis (Start, Zwischen, Ende); in EPC als Sechseck.

### **Funktion (EPC)**

Tätigkeit in der EPC. Folgt immer auf ein Ereignis. Benennung: Verb + Objekt.

### **Gateway**

Verzweigung / Zusammenführung im BPMN-Prozess. XOR (exklusiv), AND (parallel), OR (inklusive).

### **Granularität**

Tiefe eines Modells. Bewusst zu wählen, je nach Zweck und Zielgruppe.

### **GoM**

*Grundsätze ordnungsmäßiger Modellierung*. Sieben Qualitätskriterien: Richtigkeit, Relevanz, Wirtschaftlichkeit, Klarheit, Vergleichbarkeit, systematischer Aufbau, Zweckorientierung.

### **Konnektor (EPC)**

Logische Verknüpfung in der EPC: AND, OR, XOR.

### **Lane**

Innerhalb eines Pools eingerichteter Bereich für eine Rolle. Ordnet Aktivitäten einer Verantwortung zu.

### **Modellzweck**

Wofür ein Modell gebaut wird – Dokumentation, Analyse, Automatisierung, Compliance. Bestimmt Tiefe und Notation.

### **Nachrichtenfluss**

Gestrichelter Pfeil zwischen Pools in BPMN. Markiert Kommunikation über organisatorische Grenzen.

### **Organisationseinheit (EPC)**

Optionale Zuordnung in der eEPC – welche Rolle/Stelle führt die Funktion aus.

### **Pool**

Beteiligter in BPMN – typischerweise eine Organisation. Pools werden über Nachrichtenflüsse verbunden.

### **Process Mining**

Datenbasierte Rekonstruktion realer Prozesse aus Logdaten von IT-Systemen. Macht das Ist

sichtbar – als Ergänzung zum modellierten Soll. (van der Aalst, 2016)

**Prozess**

Logische Abfolge von Aktivitäten, die Input in Output transformiert und Wert erzeugt.

**RACI**

Verantwortungsmatrix: *Responsible, Accountable, Consulted, Informed*. Ergänzt Lanes um Detail-Verantwortung.

**Sequenzfluss**

Durchgezogener Pfeil zwischen Aktivitäten innerhalb eines Pools. Bestimmt die zeitliche Reihenfolge.

**Workflow-Engine**

Software, die BPMN-Modelle ausführt (z. B. Camunda, Flowable). Brücke vom Diagramm zum laufenden Prozess.

**Zweckorientierung**

Wichtigster der sieben GoM-Grundsätze. Ein Modell folgt immer einem klar benannten Zweck – alle anderen Eigenschaften ordnen sich diesem unter.